

JWT (JSON Web Token) - Complete Guide

Table of Contents

1. Introduction
2. The Problem JWT Solves
3. Session-Based Authentication
4. Problems with Sessions
5. Enter JWT
6. What is JWT?
7. Structure of JWT
 - Header
 - Payload
 - Signature
8. Types of JWT
9. JWT Authentication Flow
10. Access Token vs Refresh Token
11. Security Mechanisms
12. JWT vs Session Authentication
13. Real World Usage
14. Key Takeaways

1. Introduction

JWT stands for:

JSON Web Token

JWT is a compact and secure way of transferring information between parties.

It is primarily used for:

- Authentication
- Authorization
- Secure information exchange

2. The Problem JWT Solves

Before JWT, most applications used Session-Based Authentication.

Example:

```
Client — Login → Server
```

After successful login:

```
Server creates session  
Server stores session  
Server sends Session ID
```

Every request:

```
Client — Session ID → Server
```

Server must look up session data.

3. Session-Based Authentication

Flow

Step 1

User logs in.

```
Username + Password
```

Step 2

Server validates credentials.

Step 3

Server creates a session.

```
SessionID = S123
```

Step 4

Server stores:

S123 → Alice

Step 5

Browser stores:

SESSIONID=S123

Step 6

Future requests:

Cookie: SESSIONID=S123

Server checks session store.

Diagram

```
graph TD
    Client -- Login --> Server
    Server -- Create Session --> SessionStore[Session Store]
    SessionStore -- "S123 → Alice" --> Client
```

4. Problems with Sessions

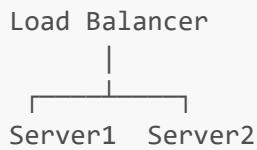
Problem 1: Scaling

Single server:

```
graph LR
    Server --- SessionStore[Session Store]
```

Works fine.

Multiple servers:



Now sessions must be shared.

Possible solutions:

- Redis
- Database

Extra complexity.

Problem 2: Performance Overhead

Every request requires:



Extra network calls.

Problem 3: Microservices

Suppose:

```
Auth Service
Order Service
Payment Service
Profile Service
```

Every service needs session access.

Managing sessions becomes difficult.

Problem 4: Mobile & APIs

Cookies are not always ideal.

Mobile apps often prefer token-based authentication.

5. Enter JWT

JWT solves these problems by being:

```
Stateless
```

The server no longer needs to store session information.

Instead:

```
Client carries identity information.
```

6. What is JWT?

JWT is a compact token format.

Structure:

```
HEADER.PAYLOAD.SIGNATURE
```

Example:

```
eyJhbGciOiJIUzI1NiIs...
```

Characteristics:

- Self-contained
- Stateless
- Signed
- Tamper-resistant

7. Structure of JWT

A JWT consists of three parts.

```
HEADER.PAYLOAD.SIGNATURE
```

Part 1: Header

Example:

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Meaning:

alg

Algorithm used for signing.

Examples:

```
HS256  
RS256  
ES256
```

typ

Token type.

```
JWT
```

Header Encoded

```
Base64Url(Header)
```

Part 2: Payload

Payload contains claims.

Example:

```
{
  "userId": 123,
  "name": "Alice",
  "role": "admin",
  "exp": 1762389012
}
```

Common Claims

sub

Subject

```
{
  "sub": "123"
}
```

name

```
{
  "name": "Alice"
}
```

role

```
{
  "role": "admin"
}
```

exp

Expiration time

```
{
  "exp": 1762389012
}
```

Important

Payload is NOT encrypted.

Anyone can decode it.

Never store:

- Passwords
- Credit cards
- Secrets

Inside payload.

Part 3: Signature

Generated using:

```
Header
+
Payload
+
Secret Key
```

Example:

```
HMACSHA256(
  Base64Url(Header)
  +
  ". "
  +
  Base64Url(Payload),
  Secret
)
```

Why Signature?

It ensures:

- Token not modified
- Token created by trusted server

8. Types of JWT

1. JWS (JSON Web Signature)

Most common.

Signed
Not Encrypted

Anyone can read payload.

Cannot modify payload.

2. JWE (JSON Web Encryption)

Encrypted

Used when payload contains sensitive information.

9. JWT Authentication Flow

Step 1

User logs in.

Username
Password

Step 2

Server validates credentials.

Step 3

Server generates JWT.

Example payload:

```
{  
  "userId": 123,  
  "role": "admin"  
}
```

Step 4

Server sends token.

```
{  
  "token": "eyJhbG..."  
}
```

Step 5

Client stores token.

Usually:

- HttpOnly Cookie
- Local Storage
- Session Storage

Preferred:

```
HttpOnly Cookie
```

Step 6

Client sends token.

```
Authorization: Bearer eyJhbG...
```

Step 7

Server verifies signature.

If valid:

```
Authenticated
```

No database lookup needed.

Diagram

```
Client
  |
  | Login
  v
Server
  |
  | Create JWT
  v
Client

Future Requests

Client
  |
  | JWT
  v
Server

Verify Signature
Authenticate User
```

10. Access Token vs Refresh Token

Modern systems usually use both.

Access Token

Short-lived.

Example:

```
15 minutes
```

Used for API access.

Refresh Token

Long-lived.

Example:

7 days
30 days

Used to obtain a new access token.

Flow

```
graph TD
    Login --> AT[Access Token]
    Login --> RT[Refresh Token]
```

Access token expires.

```
graph TD
    Client -- Refresh Token --> Server
    Server -- New Access Token --> Client
```

11. Security Mechanisms

1. HTTPS

Always use:

HTTPS

Without HTTPS:

Token can be stolen.

2. Short Expiration

Good:

15 minutes

Bad:

30 days

3. Strong Secret Key

Example:

```
JWT_SECRET=
s8D7#kL9@vQ!x2...
```

4. HttpOnly Cookies

Prevents JavaScript access.

Helps reduce XSS attacks.

5. Refresh Token Rotation

Issue a new refresh token every time.

Improves security.

12. JWT vs Session Authentication

Feature	Session	JWT
Server Stores State	Yes	No
Stateful	Yes	No
DB Lookup	Usually Yes	No
Scalability	Lower	Higher
Logout	Easy	Harder
Microservices	Harder	Easier
Mobile Apps	Less Common	Excellent

Feature	Session	JWT
APIs	Less Suitable	Excellent

13. Real World Usage

Use JWT For

- REST APIs
- React Applications
- Angular Applications
- Vue Applications
- Mobile Apps
- Microservices

Use Sessions For

- Traditional websites
- Banking systems
- Internal admin portals

14. Key Takeaways

JWT stands for:

JSON Web Token

JWT structure:

HEADER.PAYLOAD.SIGNATURE

Payload:

Readable
Not Encrypted

Signature:

Protects against tampering

JWT is:

Stateless

Benefits:

- No session storage
- Better scalability
- Great for APIs
- Great for microservices

Memory Trick:

Session = Server remembers you.

JWT = You carry proof of who you are.